Minimum-Payment Expense Settlement

November 2026

Abstract

This document describes the mathematical foundations and algorithmic methods used in the Expenses Settler project, a Python library for computing the globally minimum number of payments required to settle shared expenses from a CSV input.

The core problem reduces to computing each participant's net balance (total paid minus total owed) and then finding a minimum-size multiset of transfers (i, j, a) that zeroes all balances. We present three algorithms with different trade-offs between optimality, time, and space:

1. An **exact** bitmask dynamic programming algorithm achieving the theoretical minimum $\min(n - c, n - 1)$ payments in $O(3^n)$ time, where n is the number of non-zero-balance people and c is the maximum number of disjoint zero-sum subsets.
2. A **heuristic** greedy subset-packing algorithm running in $O(n \cdot 2^n)$ time that may miss the optimum on crafted inputs.
3. A **greedy** two-heap largest-debtor-to-largest-creditor algorithm running in $O(n \log n)$ time with no size limit but no optimality guarantee.

We prove correctness for all three, establish the $n - c$ lower bound on payment count, and provide a complexity analysis that justifies the default threshold of $n = 18$ for the exact method.

1 Problem Formulation

1.1 Input

A list of expenses, each represented as a triple (p, A, U) where:

- p is the payer (a string identifier),
- A is the total amount in non-negative integer cents,
- U is the multiset of participants sharing the expense.

1.2 Objective

Given a collection of expenses, compute the **globally minimum** set of payments – a list of triples (i, j, a) where $a > 0$ is the amount sent from person i to person j – such that after all payments, every person's net balance is zero.

1.3 Notation

- S: the set of persons with non-zero net balance, with $|S| = n$.
- A **payment** is a triple (i, j, a) with $a > 0$.
- A **settlement** is a multiset of payments P. Its size is $|P|$.
- A settlement is **feasible** if for every $i \in S$, the sum of amounts received minus the sum of amounts sent equals $-b_i$ (where b_i is the net balance: positive = owed, negative = owes).
- A settlement is **minimal** if it is feasible and no feasible settlement uses fewer payments.

2 Net Balance Computation

2.1 The Divmod Rounding Rule

For an expense (p, A, U) with $|U| = m$ participants, the share of each participant $u \in U$ is:

$$s_u = \begin{cases} \lfloor \frac{A}{m} \rfloor + 1 & \text{if } u \text{ is among the first } (A \bmod m) \text{ in CSV order,} \\ \lfloor \frac{A}{m} \rfloor & \text{otherwise.} \end{cases}$$

This implements `divmod(A, m)`: the first `(A mod m)` participants receive the extra cent each. The shares partition A exactly:

$$\sum_{u \in U} s_u = m \cdot \left\lfloor \frac{A}{m} \right\rfloor + (A \bmod m) = A.$$

2.2 Net Balance Formula

The net balance of person i after all expenses is:

$$b_i = \sum_{e:\, p_e = i} A_e \; - \sum_{e:\, i \in U_e} s_{e,i}$$

Theorem 2.1 (Zero-Sum Invariant)**.** $\sum_{i \in S} b_i = 0$.

Proof. Every expense A_e appears exactly once as a positive credit (to p_e) and exactly $|U_e|$ times with negative sign (as shares). Since the shares partition A_e exactly, the total credit minus total debit for each expense is zero. Summing over all expenses gives zero. □

2.3 Complexity

$O(E \cdot m)$ where E is the number of expenses and $m = \max |U_e|$.

3 The Settlement Problem as a Graph Problem

After computing b_i for all $i \in S$, we form a directed graph where:

- Debtors: $D = \{i \in S : b_i < 0\}$, with $d_i = -b_i$ (amount owed by i).
- Creditors: $C = \{i \in S : b_i > 0\}$, with $c_i = b_i$ (amount owed to i).

A payment (i, j, a) reduces d_i by a and c_j by a. A settlement is feasible iff all d_i and c_j reach zero.

Theorem 3.1 (The $n-1$ Bound)**.** *Any feasible settlement uses at most $n-1$ payments.*

Proof. Sort persons by balance. Let person p_1 owe the most. Send p_1's entire debt to any creditor. This eliminates at least one person. Repeat. At most $n-1$ payments needed. □

Theorem 3.2 (The $n-c$ Bound)**.** *Let c be the maximum number of pairwise disjoint zero-sum subsets of S. Then any settlement uses at least $n-c$ payments, and there exists a feasible settlement using exactly $n-c$ payments.*

Proof. **Lower bound.** Each zero-sum subset of size k involves k people. Eliminating all k internal balances requires at least $k-1$ payments (same proof as the $n-1$ bound applied to the subset). Across c disjoint subsets this accounts for $\sum(k_j - 1) = n - \sum k_j$ payments. The remaining $n - \sum k_j$ people need at most $(n - \sum k_j) - 1$ additional payments. Total $\geq n - c$.

Achievability. For each zero-sum subset of size k, apply the greedy algorithm within the subset: exactly $k-1$ payments. Summing gives $n-c$ (or $n-1$ if $n-c > n-1$, i.e., $c < 1$). □

4 The Exact Algorithm: Bitmask Dynamic Programming

4.1 High-Level Strategy

The exact algorithm achieves the theoretical minimum $n-c$ by:

1. Finding the **maximum** number c of pairwise disjoint zero-sum subsets of S.
2. For each subset, settling with greedy largest-debtor-to-largest-creditor (cost: $k-1$ for a subset of size k).
3. Settling the remainder greedily (cost: $\leq (n - \sum k_j) - 1$).

The optimal payment count equals $\min(n-c,\ n-1)$.

4.2 Step 1: Zero-Sum Mask Enumeration

Encoding. Assign each person p_j the index $j \in \{0, \ldots, n-1\}$. A subset $T \subseteq S$ is encoded as the bitmask $m \in \{0, 1, \ldots, 2^n - 1\}$ where bit j is 1 iff $p_j \in T$.

Precomputation. We precompute all zero-sum masks:

$$\texttt{zero_sum_masks} = \{m \mid \sum_{j:\text{bit } j \text{ of } m \text{ is set}} \texttt{balance_list}[j] = 0\}$$

Complexity: $O(n \cdot 2^n)$ time and space.

Observation. Since $\sum_{j=0}^{n-1} b_{p_j} = 0$ (Theorem 2), the full mask $111\ldots1$ is always zero-sum.

4.3 Step 2: Maximum Disjoint-Zero-Sum Count via DP

State. Define:

$\texttt{dp}[m]$ = the maximum number of pairwise disjoint zero-sum subsets that can be found using only people whose b

The answer is $\texttt{dp}[2^n - 1]$.

Base case. $\texttt{dp}[0] = 0$.

Transition. For each $m > 0$, let $\texttt{lowbit} = m \ \&\ (-m)$ (the least significant set bit). We consider two cases:

Option A: Do not include lowbit's person in any subset.

$$\texttt{dp}[m] = \texttt{dp}[m \oplus \texttt{lowbit}]$$

This is always feasible: if the best partition of $m \oplus \texttt{lowbit}$ uses c subsets, so does m (we just ignore the lowbit person).

Option B: Include lowbit's person in a zero-sum subset. The subset must include lowbit (otherwise covered by Option A). Iterate over all submasks $s \subseteq (m \oplus \texttt{lowbit})$, forming $\texttt{candidate} = s \mid \texttt{lowbit}$:

$$\texttt{if candidate} \in \texttt{zero_sum_masks:} \quad \texttt{dp}[m] = \max(\texttt{dp}[m],\ \texttt{dp}[m \oplus \texttt{candidate}] + 1)$$

Key insight. The person at lowbit **must** belong to some subset (if they belong to one at all). We iterate only over zero-sum subsets that **include** lowbit, avoiding redundant sub-problems and ensuring every partition is considered exactly once.

Complexity of the transition. The number of submasks of a mask of weight w is 2^{w-1}. Summing over all 2^n masks gives:

$$\sum_{w=1}^{n} \binom{n}{w} \cdot 2^{w-1} = \frac{1}{2} \sum_{w=0}^{n} \binom{n}{w} 2^w = \frac{3^n}{2} = O(3^n)$$

4.4 Step 3: Reconstruction via Parent Pointers

Maintain $\texttt{parent}[m]$ = the mask of the last zero-sum subset added in the optimal solution:

$$\texttt{parent}[m] = \begin{cases} 0 & \text{if Option A is chosen,} \\ \texttt{candidate} & \text{otherwise.} \end{cases}$$

Reconstruction algorithm:

```
selected = []
m = (1 << n) - 1
while m > 0:
    sub = parent[m]
    if sub == 0: break          # no more subsets to extract
    selected.append(people set from sub)
    m = m ^ sub                  # remove extracted subset
return selected
```

Lemma 4.1 (Correctness of reconstruction)**.** *The sequence of subsets produced by repeatedly extracting* $\texttt{parent}[m]$ *is a valid partition: extracted subsets are pairwise disjoint, all are zero-sum, and the union covers the subset of* m *for which* `parent` *was set.*

Proof. By induction on the number of extraction steps. Each iteration removes exactly $\texttt{parent}[m]$ from m, and $\texttt{parent}[m]$ is a zero-sum mask (verified during DP). Since m decreases strictly, the process terminates. The final m with $\texttt{parent} = 0$ has no subsets remaining. □

4.5 Step 4: Intra-Subset Settlement

For each discovered zero-sum subset T of size k, apply the greedy largest-debtor-to-largest-creditor within T only. Since the subset is zero-sum, this terminates with all internal balances at zero in exactly $k - 1$ payments.

4.6 Step 5: Remainder Settlement

People not in any extracted subset are settled by greedy pairwise matching over the entire balance set.

4.7 Total Payment Count

$$|P| = (k_1 - 1) + (k_2 - 1) + \cdots + (k_c - 1) + |P_{\text{remainder}}|$$

where $\sum k_j = \sum |T_j|$. If no remainder people exist: $|P| = n - c$. Otherwise $|P| \leq n - c - 1$. In all cases $|P| \leq n - 1$ (Theorem 3) and $|P| \geq n - c$ (Theorem 4).

4.8 Correctness

Theorem 4.2 (Optimality of the Exact Algorithm)**.** *The exact algorithm produces a settlement of minimum size.*

Proof. By Lemma 1, the DP finds the true maximum c. By Theorem 4, $n - c$ is a lower bound on any feasible settlement, and we construct one of that size. Therefore the result is optimal. □

4.9 Complexity Summary

Step	Time	Space
Zero-sum mask enumeration	$O(n \cdot 2^n)$	$O(2^n)$
DP computation	$O(3^n)$	$O(2^n)$
Reconstruction	$O(n \cdot c)$	$O(c)$
Greedy intra-subset	$O(n \cdot c)$	$O(n)$
Greedy remainder	$O(n \log n)$	$O(n)$
Total	$\mathbf{O(3^n)}$	$\mathbf{O(2^n)}$

Table 1: Exact algorithm complexity analysis.

The exponential bound explains the default threshold of $n = 18$: $3^{18} \approx 3.87 \times 10^8$ operations is feasible in seconds, while $3^{24} \approx 2.82 \times 10^{11}$ is infeasible.

5 The Heuristic Algorithm

5.1 Strategy

Enumerate all zero-sum subsets in order of increasing size, then greedily select non-overlapping ones:

```
for size = 2 to n:
    for each combination of size persons:
        if sum of their balances is zero:
            add to zero_sum_subsets

sort zero_sum_subsets by size (smallest first)
selected = []; used = empty set
for subset in zero_sum_subsets:
```

```
    if subset \\cap used == \\emptyset:
        selected.append(subset)
        used |= subset
```

5.2 Complexity

- Enumeration: $\sum_{k=2}^{n} \binom{n}{k} = 2^n - n - 1$ subsets. Each check costs $O(n)$. Total: $O(n \cdot 2^n)$.
- Sorting: $O(n^2 \cdot 2^n)$.
- Greedy selection: $O(n \cdot 2^n)$.
- **Dominant term:** $O(n \cdot 2^n)$.

5.3 Not Guaranteed Optimal

The heuristic may select a large subset that blocks two smaller disjoint subsets. For example, with balances chosen such that a size-3 subset consumes people needed for two disjoint size-2 subsets, the heuristic produces more total payments than the exact DP.

6 The Greedy Algorithm: $O(n \log n)$

6.1 Strategy

Settle all balances in a single greedy pass using two min-heaps:

- **Debtors heap:** keyed by $(-b_i, i)$, so the smallest key = the largest debtor.
- **Creditors heap:** keyed by $(-b_i, i)$, so the smallest key = the largest creditor.

Why negate. Python's `heapq` is a min-heap. Negating the balance value makes the smallest heap entry correspond to the largest absolute balance.

6.2 Algorithm

```
debtors = minheap[(-b_i, i) for i if b_i < 0]
creditors = minheap[(-b_i, i) for i if b_i > 0]

while debtors and creditors:
    k_d, d = debtors[0]      # k_d = -b_d < 0
    k_c, c = creditors[0]    # k_c = -b_c < 0
    amount = min(-k_d, -k_c)
    emit payment(d, c, amount)
    pop both heaps
    push back d if balance not exhausted
    push back c if balance not exhausted
```

Termination. Each iteration reduces the sum of remaining balances by `amount` > 0. The sum is finite and non-negative, so the process terminates.

Correctness. At termination, at most one heap is non-empty. If both are non-empty, the heap-invariant (smallest = largest) implies a positive balance exists in both, so another payment can be made – contradiction. Exactly one heap is non-empty. But since $\sum b_i = 0$ (Theorem 2), the remaining balances must all be zero.

6.3 Payment Count and Gaps

Each iteration that fully clears both a debtor and a creditor removes one person from consideration (one payment per removal). The upper bound is $n - 1$ payments (at least one person removed per payment).

Not guaranteed minimal. In multi-group scenarios, the greedy may bridge zero-sum groups that could be settled more efficiently in isolation. The gap between greedy and optimal can be arbitrarily large for sufficiently structured inputs (multiple disjoint zero-sum components connected through a single "bridge" person).

6.4 Complexity

- Heap construction: $O(n)$.
- Each iteration pops one person and pushes back at most one. Total iterations $\leq 2n - 1$.
- Each push/pop: $O(\log n)$.
- **Total:** $O(n \log n)$ time, $O(n)$ space.

7 Comparison Summary

Property	Exact	Heuristic	Greedy
Optimality	✓ provably optimal	× not guaranteed	× not guaranteed
Time	$O(3^n)$	$O(n \cdot 2^n)$	$O(n \log n)$
Space	$O(2^n)$	$O(n \cdot 2^n)$	$O(n)$
Size limit	$n \leq 18$ (default)	any n	any n
Method	Bitmask DP	Greedy subset packing	Two-heap pairwise

Table 2: Algorithm comparison.

8 Edge Cases and Invariants

8.1 Zero-Value Expenses

An expense with $A = 0$ contributes 0 to every participant's share. No effect on balances.

8.2 Single Participant

An expense with $|U| = 1$ credits the payer A and debits the participant A by A. Net: $b_{\text{payer}} \leftarrow b_{\text{payer}} + 0$. No effect.

8.3 Payer Not a Participant

The payer receives $+A$ (credited) but has no negative share. Net: $b_{\text{payer}} \leftarrow b_{\text{payer}} + A$. The payer is a creditor for that expense.

8.4 Payer Is a Participant

The payer receives $+A$ and a negative share s_p. Net: $b_p \leftarrow b_p + A - s_p$. Since $s_p \leq A/m < A$ when $|U| \geq 2$, the payer always has a positive net from their own expense.

8.5 Invariants

1. **All payment amounts are positive.** Every payment amount is $\min(-d_{\text{debtor}}, c_{\text{creditor}})$ with both arguments strictly positive (we only consider non-zero balances).

2. **No self-payment.** The debtor heap and creditor heap are disjoint sets (balance < 0 vs. balance > 0). No person can appear in both.

9 Source Reference

All algorithms are implemented in `settle_expenses.py`:

Function	Lines	Role
`net_balances()`	§2	Compute per-person net balance from expenses
`settle_expenses()`	§3	Main entry; dispatches to exact/heuristic/greedy
`_settle()`	§4	Greedy largest-debtor-to-largest-creditor within a group
`_greedy_settle_balances()`	§6	Polynomial $O(n \log n)$ two-heap settlement
`_find_max_zero_sum_subsets_exact()`	§4	Bitmask DP, optimal subset count
`_find_max_zero_sum_subsets_heuristic()`	§5	Greedy subset enumeration, non-optimal

10 Dispatch Logic

The `threshold` parameter of `settle_expenses()` controls when "auto" falls back:

$$\text{effective method} = \begin{cases} \texttt{exact} & \text{if } \texttt{method} = \texttt{"exact"} \\ \texttt{greedy} & \text{if } \texttt{method} = \texttt{"greedy"} \\ \texttt{greedy} & \text{if } \texttt{method} = \texttt{"auto"} \text{ and } n > \texttt{threshold} \\ \texttt{exact} & \text{if } \texttt{method} = \texttt{"auto"} \text{ and } n \leq \texttt{threshold} \end{cases}$$

With default `threshold` $= 18$, the exact DP is used up to 18 people, and a `UserWarning` is emitted when falling back.